Univ.-Prof. Dr. Daniel Große
DI Simon Reitinger & Katharina Ruep, M.Sc.

12. März 2026

Übung 1
zur Vorlesung Rechnerarchitektur

Abgabe bis **Donnerstag 19 März, 2026 10:15** via EPICS: `https://ep.ics.jku.at`.

RISC-V Programme müssen in der ICS Version von *Venus*[1] ausführbar sein. Zur Ausarbeitung der Übung darf nur der `RV32I` Befehlssatz mit der `M` Erweiterung verwendet werden. **Pseudobefehle sind (soweit nicht explizit erlaubt) nicht zu verwenden.** Die Lösungen sollten ausreichend kommentiert werden, im Speziellen sind Annahmen über die Registerbelegung der Variablen anzugeben.

Aufgabe 1 – Codefragmente und Datentypen (3 + 3 + 2 Punkte)

Übersetze die folgenden C - Codefragmente in RISC-V Assembler.

a) Ermittle und gib hier zusätzlich an, wie sich die Shift-Operationen auf den Integer-Datentypen auswirken. Speichere das Ergebnis y in Register $a0$.

```
int a = -7;
int b = 11;
int y = ((a >> b) - (-(a << 3))) ^ 512;
```

b) Speichere das Ergebnis `y` in Register `a0`.

```
int a = 0xdeadbeef;
int b = 0x0badc0de;
int y = (~a & ~b) | (a ^ b);
```

c) Ermittle und erkläre anhand der RISC-V Calling Convention [2] den Unterschied zwischen den Datentypen `unsigned int` und `unsigned long`.

[1] Online Simulator unter `https://venus.ics.jku.at/`.
[2] `https://riscv.org/wp-content/uploads/2024/12/riscv-calling.pdf`

Aufgabe 2 – Größte Fibonacci-Zahl in 32-Bit (8 Punkte)

Schreibe ein RISC-V Assembler Programm, das die größtmögliche Fibonacci-Zahl Y findet, welche in 32-Bit (unsigend) dargestellt werden kann und gib zusätzlich den Index k zurück.
Definition:

$$F_k = \begin{cases} 0, & k = 0 \\ 1, & k = 1 \\ F_{k-1} + F_{k-2}, & k > 1 \end{cases}$$

a) Speichere die größtmögliche Fibbonacci-Zahl Y in `a0` und den Index k in `a1`, so dass gilt:

$$F_k = Y \quad \text{und} \quad Y < 2^{32}.$$

b) Gib an, warum ein einfacher Vergleich mittels `bltu` zwischen F_k und `MAX_VAL`$(2^{32} - 1)$ nicht möglich ist.

Aufgabe 3 – Array-Zugriffe (8 Punkte)

Gegeben ist folgender RISC-V Assembler Code:

```
.globl main

.data
arr: .word 0x4, 0x8, 0x12, 0x11, 0x5, 0x21, 0x1, 0x99, 0x41, 0xFF, 0xAB
n: .word 11

.text
main:
```

Schreibe ein Programm, welches die Elemente des Arrays `arr` "in-place" umkehrt, d.h. das Array wird direkt im Speicher verändert, ohne ein zweites Hilfsarray zu verwenden. Zusätzlich soll in der selben Schleife die Summe aller Elemente des Arrays `arr` in Register $a0$ gespeichert werden.
Beispiel:
vor dem Programm: `arr` = 1 4 8 9
nach dem Programm: `arr` = 9 8 4 1; `a0` = 22

Hinweise: Die Verwendung des Pseudobefehls *la* ist in dieser Aufgabe erlaubt.

Aufgabe 4 – (Bonus) Little-Endian vs. Big-Endian (2 Punkte)

Das folgende RISC-V Programm ist gegeben:

```
lui  t0, 0x12345
addi t0, s3, 0x678
sw   t0, 0(a0)
lb   t1, 1(a0)
```

Welchen Wert enthält $t1$ auf einem Little-Endian-System und welchen auf einem Big-Endian-System?
Erkläre, wie dieses Programm genutzt werden kann um herauszufinden, ob ein Computer Little-Endian oder Big-Endian verwendet.